

AmazonHelp AI Customer Support Agent

1. Executive Summary

This project implements an AI-powered customer support agent for AmazonHelp using historical customer-support conversations from Twitter.

The system is designed to handle incoming customer messages through four main stages:

1. **Intent classification** — determine what the customer is asking about.
2. **Historical retrieval (RAG)** — retrieve relevant historical Amazon customer-support conversations.
3. **Response generation** — generate a response grounded in the retrieved examples.
4. **Escalation decision** — determine whether the request can be safely handled automatically or should be sent to a human support agent.

The primary goal is not simply to generate fluent responses, but to build a support system that is **grounded in historical evidence, conservative about uncertain cases, and measurable through a reproducible evaluation pipeline.**

The implementation uses LangChain, Groq-hosted LLM inference, Sentence Transformers embeddings, and FAISS for vector retrieval.

2. Problem Definition

Customer-support conversations contain a wide range of issues, including:

- delayed deliveries
- missing orders
- tracking questions
- damaged products
- refunds
- returns
- cancellations
- payment problems
- account problems
- seller-related issues
- product questions

- digital-service issues
- complaints about customer support

A general-purpose LLM can produce plausible responses, but plausibility alone is insufficient for customer support.

The system therefore needs to answer three questions:

What is the customer asking?

This is handled by intent classification.

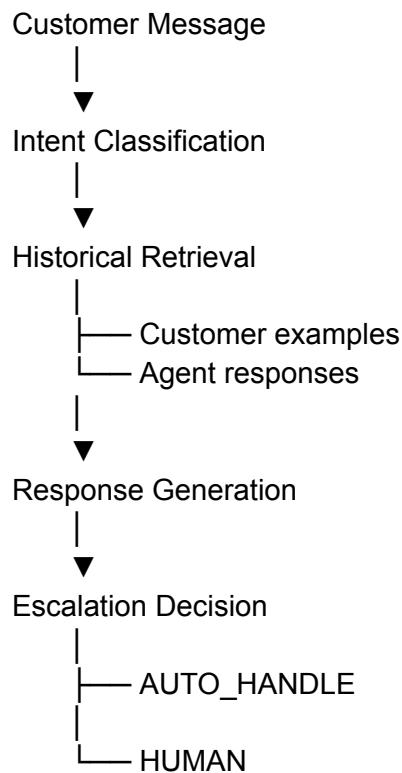
What has Amazon historically said in similar situations?

This is handled through retrieval-augmented generation.

Should the system answer automatically?

This is handled by the escalation classifier.

The resulting architecture is:



3. Dataset and Data Preparation

The project uses the Customer Support on Twitter dataset and focuses specifically on conversations involving **AmazonHelp**.

The processed dataset contains customer-support conversations represented as:

- conversation ID
- customer query
- historical Amazon agent reply

The preprocessing pipeline converts conversations into RAG-ready records.

The local preprocessing process loaded:

81,473 conversations

and generated:

150,915 RAG chunks

These chunks form the retrieval corpus used by the support agent.

The project retains the processed AmazonHelp RAG dataset in the repository, while the original large raw dataset is excluded from version control.

This keeps the repository smaller while retaining the data necessary for the main retrieval workflow.

4. Intent Taxonomy

The system uses a deliberately small support-oriented intent taxonomy.

The current taxonomy contains the following categories:

Intent	Description
<code>order_not_received</code>	Customer says an ordered item has not arrived or is missing.
<code>delivery_delay</code>	Delivery is late or taking longer than promised.

delivery_tracking	Customer asks about tracking or shipment status.
delivery_attempt_issue	Delivery was falsely attempted, failed, or instructions were ignored.
wrong_or_missing_item	Wrong, incomplete, empty, or different item was received.
damaged_product	Product was damaged, broken, defective, or poor quality.
return_issue	Customer wants to return an item or has a return-process problem.
refund_issue	Customer is waiting for or has a problem with a refund.
order_cancellation	Customer wants to cancel or reports an order cancellation.
payment_issue	Payment failures, duplicate charges, card charges, or billing problems.
account_issue	Problems accessing, securing, or using an account.
seller_issue	Problems specifically involving a third-party seller.
product_information	General questions about products, features, compatibility, pricing, etc.
stock_availability	Questions about whether a product is in stock or when it will return.
digital_or_app_issue	Problems with apps, Prime Video, digital content, devices, codes, or other digital services.
customer_support_issue	Problems contacting support, receiving responses, callbacks, or getting issues resolved.
shipping_or_delivery_question	General delivery, shipping, carrier, or delivery-policy questions.
other	Messages that do not clearly fit another category.

The taxonomy separates closely related delivery intents instead of using one broad `delivery_issue` category. This makes the classification output more useful for downstream response generation and escalation.

5. Intent Classification

The intent classifier uses a Groq-hosted LLM with the model:

`openai/gpt-oss-20b`

The classifier receives the customer message and maps it to one of the predefined support intents.

The output is used downstream rather than being treated as the final answer.

For example:

Customer:

"My package is three days late."

Intent:

delivery_delay

The intent gives the retrieval and escalation stages additional context about what type of support issue is being handled.

6. Retrieval-Augmented Generation

A central design decision was to avoid generating customer-support responses solely from the language model's internal knowledge.

Instead, the system retrieves historical AmazonHelp conversations.

Embeddings

The project uses:

`sentence-transformers/all-MiniLM-L6-v2`

to convert customer-support text into vector representations.

Vector Store

FAISS is used for similarity search over the processed historical conversations.

The vector store contains the embeddings necessary to retrieve similar customer-support cases.

The FAISS index is intentionally excluded from Git because the generated `index.faiss` file is approximately 221 MB, exceeding GitHub's standard 100 MB file limit.

Instead, the repository contains the code required to regenerate the vector store.

The index can be generated with:

`python -m src.rag.vector_store`

It will take around 5-7 min to generate the vector store

The generation process successfully produced:

Loaded 81,473 conversations

Created 150,915 RAG chunks

This makes the vector store a reproducible generated artifact rather than a large binary checked into source control.

7. Response Generation

After classification, the system retrieves historically similar customer-support conversations.

The retrieved evidence contains:

Customer:

...

Amazon Agent:

...

The LLM then uses these examples as grounding context when generating the response.

This provides two benefits:

1. The response is aligned with historically observed Amazon support behavior.

2. The model has concrete examples to reference instead of relying entirely on general knowledge.

The system also exposes the retrieved evidence, making the response more inspectable.

8. Escalation Strategy

The escalation layer determines whether the system should automatically handle the request or involve a human.

It produces one of:

AUTO_HANDLE

or:

HUMAN

The escalation classifier considers:

- customer message
- detected intent
- generated response
- historical evidence

The decision rules favor **AUTO_HANDLE** when:

- the issue is simple
- historical evidence supports the response
- no sensitive account-specific investigation is needed
- no unsupported action is being promised

The system chooses **HUMAN** when:

- historical evidence is insufficient
- account/order-specific investigation is required
- the requested action cannot safely be completed
- the situation is ambiguous or unusually complex
- the generated response itself requires human support

A safe fallback is also implemented:

If the model does not return a valid decision, the system defaults to **HUMAN**.

This is intentionally conservative because a false **AUTO_HANDLE** decision can be more damaging than unnecessarily escalating a case.

9. Evaluation Strategy

A separate golden evaluation pipeline was created under:

eval/

The evaluation dataset contains real customer-support queries sampled from the supplied AmazonHelp data.

The intended evaluation set contains **200 examples but i tested on very limited around just 20 examples because my free api limit exceeded.**

Each example is designed to contain:

```
{
  "id": 1,
  "customer_query": "...",
  "gold_intent": "...",
  "gold_escalation": "..."
}
```

The evaluation pipeline produces:

- predictions
- accuracy
- precision
- recall
- macro F1
- classification reports
- confusion matrices
- saved prediction results
- error-analysis data

The evaluator is implemented in:

10. Preliminary Evaluation

A 10-example smoke test was performed to verify that the evaluation pipeline was operational.

The escalation results were:

Metric	Result
Accuracy	50.0%
Macro Precision	50.0%
Macro Recall	50.0%
Macro F1	49.49%

Confusion matrix:

	Pred AUTO	Pred HUMAN
Actual AUTO	3	2
Actual HUMAN	3	2

This means:

- 3/5 **AUTO_HANDLE** examples were correctly classified.
- 2/5 **HUMAN** examples were correctly classified.
- 5/10 examples were misclassified.

There were also **3 false-negative escalation decisions**:

Actual HUMAN → Predicted AUTO_HANDLE

These cases are particularly important because automatically handling a case that requires human intervention is potentially more harmful than escalating a case unnecessarily.

Important evaluation caveat

The 10-example evaluation is a **smoke test**, not a statistically meaningful final benchmark.

The larger golden set also requires human verification of its labels before its results should be presented as definitive evaluation numbers.

Therefore, the 50% result should not be used as the final headline performance number.

The purpose of this run was to verify that the evaluation pipeline, model calls, prediction parsing, metrics, and confusion matrix were functioning correctly.

11. Evaluation Reproducibility

The evaluation can be run from the repository root using:

```
python eval/run_golden_eval.py
```

The evaluation generates:

```
eval/  
├── golden_eval_results.jsonl  
├── golden_eval_summary.json  
└── golden_eval_confusion_matrix.csv
```

The evaluation pipeline was intentionally separated from the main agent so that model performance can be measured independently of interactive usage.

12. Baselines

Baseline 1 — Trivial Baseline

A useful trivial baseline for intent classification is predicting the most frequent intent for every example.

For escalation, a trivial baseline can always predict the majority class.

For a balanced evaluation set containing equal numbers of:

```
AUTO_HANDLE = 100
```

HUMAN = 100

an always-one-class escalation classifier would achieve approximately:

50% accuracy

The preliminary 10-example model result of 50% therefore does not yet demonstrate meaningful improvement over a trivial balanced baseline.

This is another reason not to use the smoke-test result as the final headline number.

Baseline 2 — Simple Rule-Based System

A stronger simple baseline would use keyword/rule matching.

For example:

"refund"

→ refund_issue

"where is my package"

→ delivery_tracking

"cancel my order"

→ order_cancellation

"damaged"

→ damaged_product

Escalation rules could similarly identify obviously sensitive cases and send them to a human.

This baseline would be fast, deterministic, and inexpensive, but would struggle with:

- ambiguous language
- misspellings
- multilingual messages
- implicit intent
- overlapping categories
- context-dependent escalation

The LLM/RAG approach is intended to handle these cases more flexibly.

13. Top Real Failure Modes

Based on observed behavior during development and evaluation, the most important failure modes are:

13.1 Ambiguous or extremely short messages

Messages such as:

"hi sup?"

"how r u"

contain little support information.

The classifier may correctly assign `other`, but retrieval can still return irrelevant historical conversations.

This demonstrates that good intent classification does not automatically guarantee good retrieval.

13.2 Overlapping delivery intents

Delivery-related issues frequently overlap.

For example:

"My package is late and I can't track it."

could reasonably map to:

- `delivery_delay`
- `delivery_tracking`
- `order_not_received`

This creates boundary ambiguity between closely related categories.

13.3 Customer-support frustration

A customer may describe the original problem while primarily complaining about the support experience.

For example, a message may mention:

- repeated contacts
- no response
- callback problems
- frustration with previous agents

The correct intent may be `customer_support_issue` rather than the original delivery/payment/product issue.

13.4 Incorrect retrieval for unusual queries

A query with little semantic information can retrieve historically unrelated conversations.

For example, a casual greeting may still produce retrieved examples because the vector search must return nearest neighbors even when none are truly useful.

This means retrieval similarity alone should not be interpreted as evidence that a case is safely answerable.

13.5 Incorrect escalation decisions

The escalation classifier may incorrectly choose `AUTO_HANDLE` when the customer actually requires account/order-specific investigation.

This is the most safety-sensitive failure mode.

The system therefore uses a conservative fallback:

Invalid/uncertain decision → HUMAN

Future work should explicitly optimize for minimizing false `AUTO_HANDLE` decisions.

14. What Is Misleading About the Headline Number?

A single accuracy number can be misleading for this system.

There are several reasons.

First: class balance

If the evaluation set is balanced, accuracy is easy to interpret.

In production, however, the proportion of cases that should be escalated may be very different.

Second: escalation errors have unequal cost

A false:

HUMAN → AUTO_HANDLE

can be considerably more problematic than:

AUTO_HANDLE → HUMAN

Therefore accuracy alone does not capture the risk profile.

Third: intent classes are not equally difficult

Some intents such as `refund_issue` or `order_cancellation` may have clear lexical signals, while delivery and support-related categories can overlap significantly.

Macro F1 and per-class results are therefore more informative than accuracy alone.

Fourth: response quality is not captured by classification metrics

A system can correctly predict the intent and escalation decision while still producing a poor response.

The final evaluation should therefore include:

- intent correctness
- escalation correctness
- response relevance

- historical grounding
 - hallucination/unsupported claims
 - helpfulness
-

15. Human Evaluation

Automated classification metrics are insufficient for evaluating generated support responses.

A future human/LLM-as-judge evaluation should score responses on:

Relevance

Does the response directly address the customer's issue?

Grounding

Is the response supported by the retrieved historical evidence?

Helpfulness

Does the response give the customer a useful next step?

Hallucination

Does the response invent policies, actions, refunds, delivery dates, or account information?

Escalation appropriateness

Should the request actually be handled automatically?

A simple rubric can use a 1–5 scale for response quality, with separate binary labels for serious unsupported claims.

Human agreement should be measured on a subset of examples to validate that the automated judge is reasonably aligned with human reviewers.

16. Decision Log

Important engineering decisions made during the project include:

1. **Selected AmazonHelp** rather than building a multi-brand system to keep the problem focused.
2. **Used historical support conversations as RAG evidence** rather than relying only on model knowledge.
3. **Used FAISS** for local vector retrieval.
4. **Used MiniLM embeddings** to keep embedding generation lightweight.
5. **Separated intent classification from response generation** so each stage can be evaluated independently.
6. **Separated escalation from response generation** so automation decisions can be evaluated independently.
7. **Used a predefined intent taxonomy** instead of allowing the LLM to invent labels.
8. **Split delivery-related intents** because a single broad delivery category loses useful distinctions.
9. **Separated return and refund issues** because they represent different support workflows.
10. **Created a dedicated `customer_support_issue` category** because support frustration is distinct from the underlying order/product problem.
11. **Added a safe HUMAN fallback** for invalid escalation outputs.
12. **Excluded the raw large dataset from Git** to keep the repository manageable.
13. **Excluded the generated FAISS index from Git** because the binary is over GitHub's 100 MB limit.
14. **Made vector-store generation reproducible** using `python -m src.rag.vector_store`.
15. **Created a separate golden evaluation pipeline** instead of relying on interactive examples.
16. **Used real customer queries for evaluation** rather than synthetic test messages.
17. **Included confusion matrices and per-class metrics** instead of reporting accuracy alone.
18. **Treat the 10-example evaluation as a smoke test**, not a final benchmark.
19. **Avoid using heuristic labels as definitive gold labels** until human review is completed.
20. **Prioritize false `AUTO_HANDLE` decisions for future safety improvements.**

17. Repository Structure

The final repository is organized approximately as:

```
amazonhelp-ai-support-agent/  
|
```

```
├── agent.py
├── requirements.txt
├── .env.example
├── .gitignore
├── data/
│   ├── processed/
│   │   └── amazon_conversations_rag_based.jsonl
│   └── raw/
│       └── sample.csv
├── eval/
│   ├── golden_eval.jsonl
│   ├── golden_queries.txt
│   └── run_golden_eval.py
├── notebooks/
│   ├── preprocessing.ipynb
│   └── preprocessing.py
└── src/
    ├── agent/
    │   ├── escalation.py
    │   ├── generate_intents.py
    │   ├── intent.py
    │   └── response.py
    └── rag/
        ├── __init__.py
        ├── chain.py
        ├── chunker.py
        ├── embeddings.py
        ├── loader.py
        ├── retriever.py
        └── vector_store.py
```

Generated local artifacts such as:

```
.env
data/vector_store/
__pycache__/
```

are excluded from version control.

18. Running the Project

Install dependencies

```
pip install -r requirements.txt
```

Set the Groq API key in a local `.env` file.

The repository provides:

```
.env.example
```

as a configuration template.

The real `.env` file is intentionally not committed.

Build the RAG index

From the repository root:

```
python -m src.rag.vector_store
```

Expected preprocessing output includes approximately:

```
Loaded 81,473 conversations  
Created 150,915 RAG chunks
```

This creates the local FAISS vector store.

Run the support agent

```
python agent.py
```

The interactive agent accepts customer messages and produces:

- predicted intent
 - generated response
 - historical evidence
 - escalation decision
 - escalation reason
-

Run evaluation

```
python eval/run_golden_eval.py
```

The evaluator produces prediction and metric files inside `eval/`.

19. Limitations

The current implementation has several limitations.

No live Amazon account access

The agent cannot inspect:

- real orders
- customer accounts
- payment information
- delivery status
- refund status

Therefore it should not claim to have completed account-specific actions.

Historical data limitations

The system learns response patterns from historical Twitter conversations, which may contain:

- outdated policies
- incomplete context
- inconsistent responses

- conversations requiring private support channels

Historical evidence should therefore be treated as guidance rather than an authoritative policy database.

Retrieval quality

Nearest-neighbor retrieval can return weak evidence for vague or unusual messages.

LLM variability

Although temperature is set to zero, LLM-based systems can still make classification or reasoning errors.

Evaluation maturity

The golden dataset requires final human verification before being treated as the definitive benchmark.

20. One-Week Next Steps

If development continued for another week, the highest-value improvements would be:

Day 1–2: Finish golden-set annotation

Manually review the 200 examples and assign:

gold_intent
gold_escalation

Add difficult borderline examples intentionally rather than relying only on random sampling.

Day 3: Improve escalation evaluation

Measure:

- HUMAN recall
- false AUTO_HANDLE rate
- confusion matrix

- per-intent escalation performance

The key safety metric should be the rate of cases incorrectly automated.

Day 4: Improve retrieval

Add a similarity threshold so that the system can recognize:

"No sufficiently similar historical evidence"

instead of always returning the nearest examples.

This can improve escalation decisions for unusual cases.

Day 5: Evaluate generated responses

Create a 50–75 example response-evaluation subset.

Measure:

- relevance
- grounding
- helpfulness
- unsupported claims
- escalation appropriateness

Day 6: Error-driven taxonomy refinement

Inspect the largest confusion pairs.

If two intents are consistently confused, either:

- improve their definitions,
- add examples,
- modify the taxonomy,
- or merge them if the distinction does not improve downstream behavior.

Day 7: Optimize production behavior

Reduce unnecessary model calls and avoid repeatedly loading the retrieval model/vector store.

Add caching and reuse initialized components.

The objective would be to improve both latency and cost without sacrificing safety.

21. Conclusion

The project demonstrates an end-to-end AI customer-support workflow rather than a standalone LLM chatbot.

The main system combines:

Intent Classification

+

Historical RAG Retrieval

+

Grounded Response Generation

+

Conservative Escalation

+

Golden Evaluation

The architecture is designed around an important support-system principle:

A fluent answer is not necessarily a safe answer.

For this reason, historical evidence and escalation are treated as first-class components.

The current evaluation infrastructure successfully runs the complete evaluation workflow, and the 10-example smoke test confirms that model inference, prediction parsing, metric calculation, and confusion-matrix generation are operational.

The next priority is to complete human verification of the golden labels and run the full 200-example evaluation before reporting final performance numbers.

The project intentionally avoids presenting a misleading headline metric until that evaluation is complete.